PML -Cricket Object Detection Project –Group 27

CMInDS
Indian Institute of Technology Bombay

Group Members

1. Yugal Sahu
2. Pravin Patil
3. Yogesh Gupte
4. Ratish Nair
5. Gaurav Patil

Objective:

Develop a machine learning model to detect cricket objects (bat, ball, stumps, or no object) in images using classical ML techniques and manual feature engineering.

Scope & Approach:

- Created dataset of cricket images (balanced across object classes)
- Preprocessed images to 800×600 resolution and divided into 8×8 grid cells
- Annotated each cell with object labels (0: no object, 1: ball, 2: bat, 3: stump)
- Extracted handcrafted features (HOG, grayscale) for classification
- Trained and evaluated Random Forest and KNN models

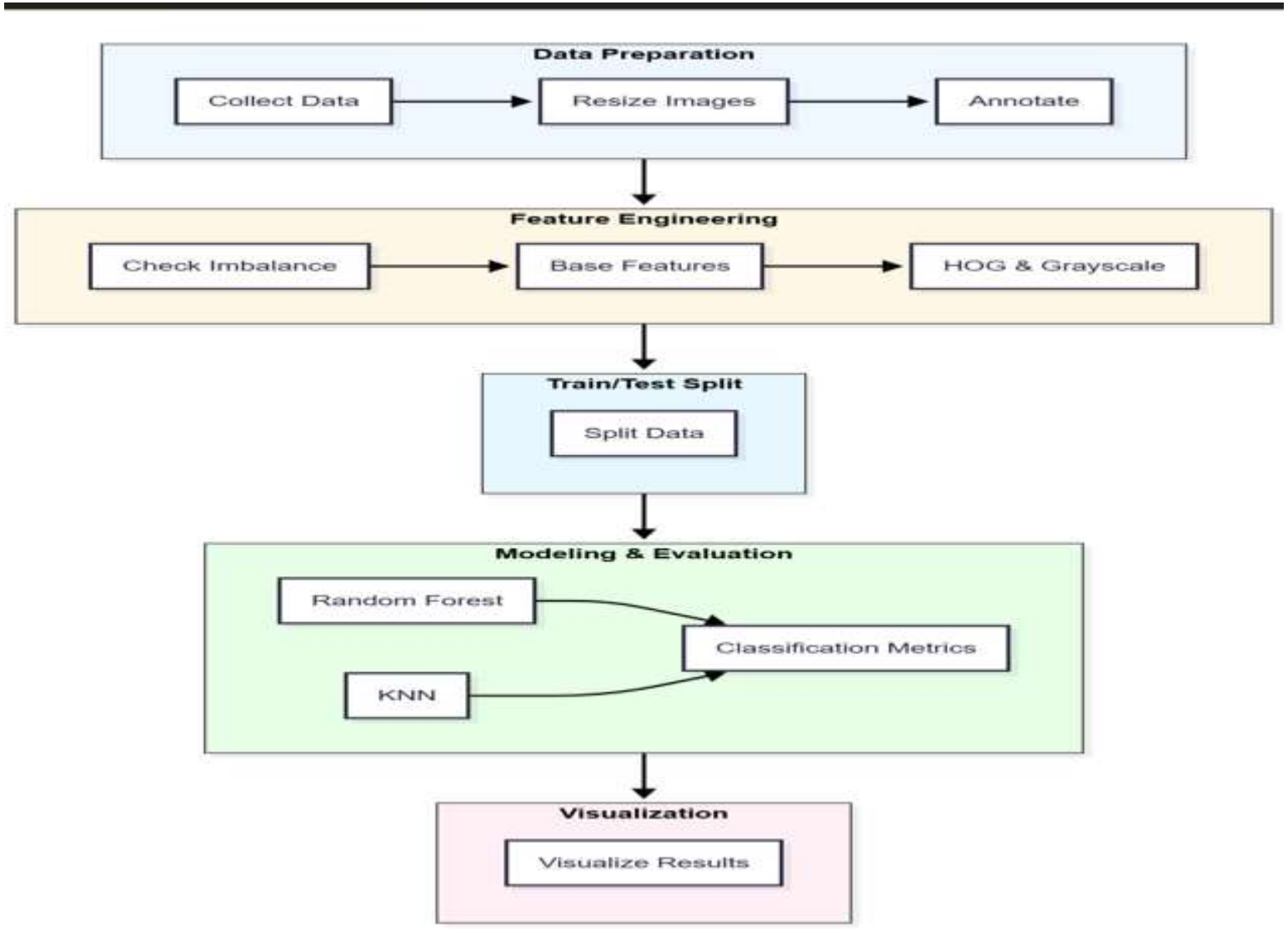
Key Deliverables:

- Source code
- Predictions CSV
- Performance metrics & visualizations
- Presentation & video

Evaluation Criteria:

- Problem detailing & solution approach
- Model performance on train/test
- Quality of documentation and presentation

Step-by-Step Workflow



Data Collection & Pre Processing

CMInDS
Indian Institute of Technology Bombay

- Gathered various types of cricket images, balanced across object classes.
- All images resized to 800×600 and partitioned into an 8×8 grid.
- Used python script to resize and reshape all images

```python
def crop_to_aspect_ratio(image: Image.Image, aspect_width: int = 4,
                         aspect_height: int = 3) -> Image.Image:
    """
    Crop an image to the specified aspect ratio (center crop).

    Args:
        image: PIL Image object
        aspect_width: Width aspect ratio
        aspect_height: Height aspect ratio

    Returns:
        Cropped PIL Image object
    """
    width, height = image.size
    target_ratio = aspect_width / aspect_height
    current_ratio = width / height

    if abs(current_ratio - target_ratio) < 0.001:
        # Already correct aspect ratio
        return image

    if current_ratio > target_ratio:
        # Image is too wide, crop width
        new_width = int(height * target_ratio)
        left = (width - new_width) // 2
        return image.crop((left, 0, left + new_width, height))
    else:
        # Image is too tall, crop height
        new_height = int(width / target_ratio)
        top = (height - new_height) // 2
        return image.crop((0, top, width, top + new_height))
```

```python
try:
    image = Image.open(image_path)
    # Convert to RGB if necessary
    if image.mode != 'RGB':
        image = image.convert('RGB')
    # Check minimum size (before any cropping)
    width, height = image.size

    # For non-4:3 images, we need to check if we can get 800x600 after cropping
    target_ratio = 4 / 3
    current_ratio = width / height
    if current_ratio > target_ratio:
        # Will crop width - check height is sufficient
        if height < IMAGE_HEIGHT:
            return None
        potential_width = int(height * target_ratio)
        if potential_width < IMAGE_WIDTH:
            return None
    else:
        # Will crop height - check width is sufficient
        if width < IMAGE_WIDTH:
            return None
        potential_height = int(width / target_ratio)
        if potential_height < IMAGE_HEIGHT:
            return None

    # Crop to 4:3 aspect ratio
    image = crop_to_aspect_ratio(image)

    # Validate final size
    if not validate_image_size(image):
        return None

    # Resize to target dimensions
    image = resize_image(image)
```

Annotation

- Explored tools (LabelImg, Copilot-assisted tools) but they were not adequate for our grid-cell tagging needs.
- Reused existing utility which was created by our peers and extended it as custom annotation utility to perform manual per-cell tagging.
- Utility produces one annotation text file per image (compatible with our use case).

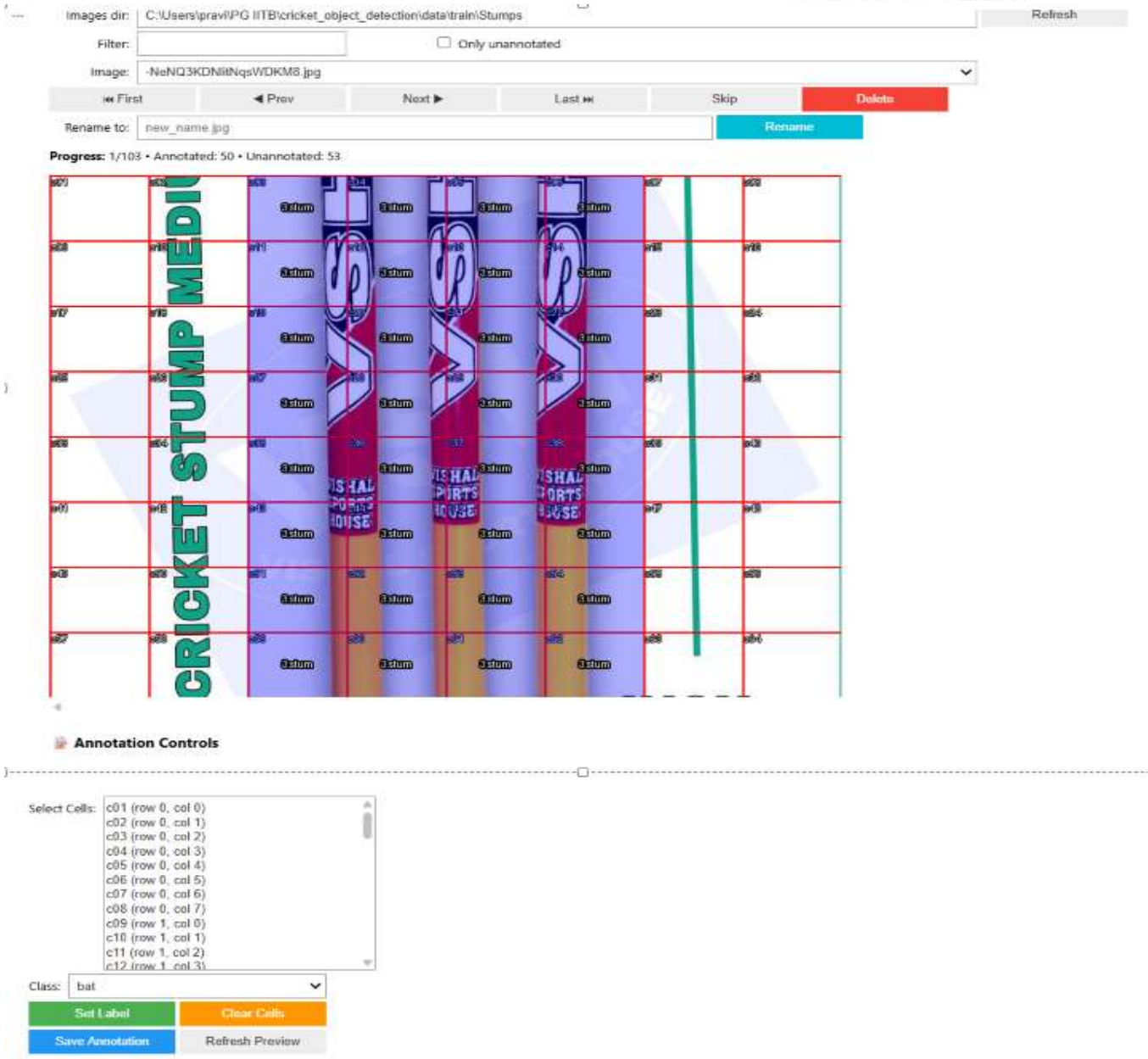
Annotation Output

cricket_object_detection > data > annotations_new > 1-One-Cricket-Bats.txt > data

```
1  c01,0
2  c02,0
3  c03,2
4  c04,2
5  c05,2
6  c06,2
7  c07,0
8  c08,0
9  c09,0
10 c10,0
11 c11,2
12 c12,2
```

Annotation Utility

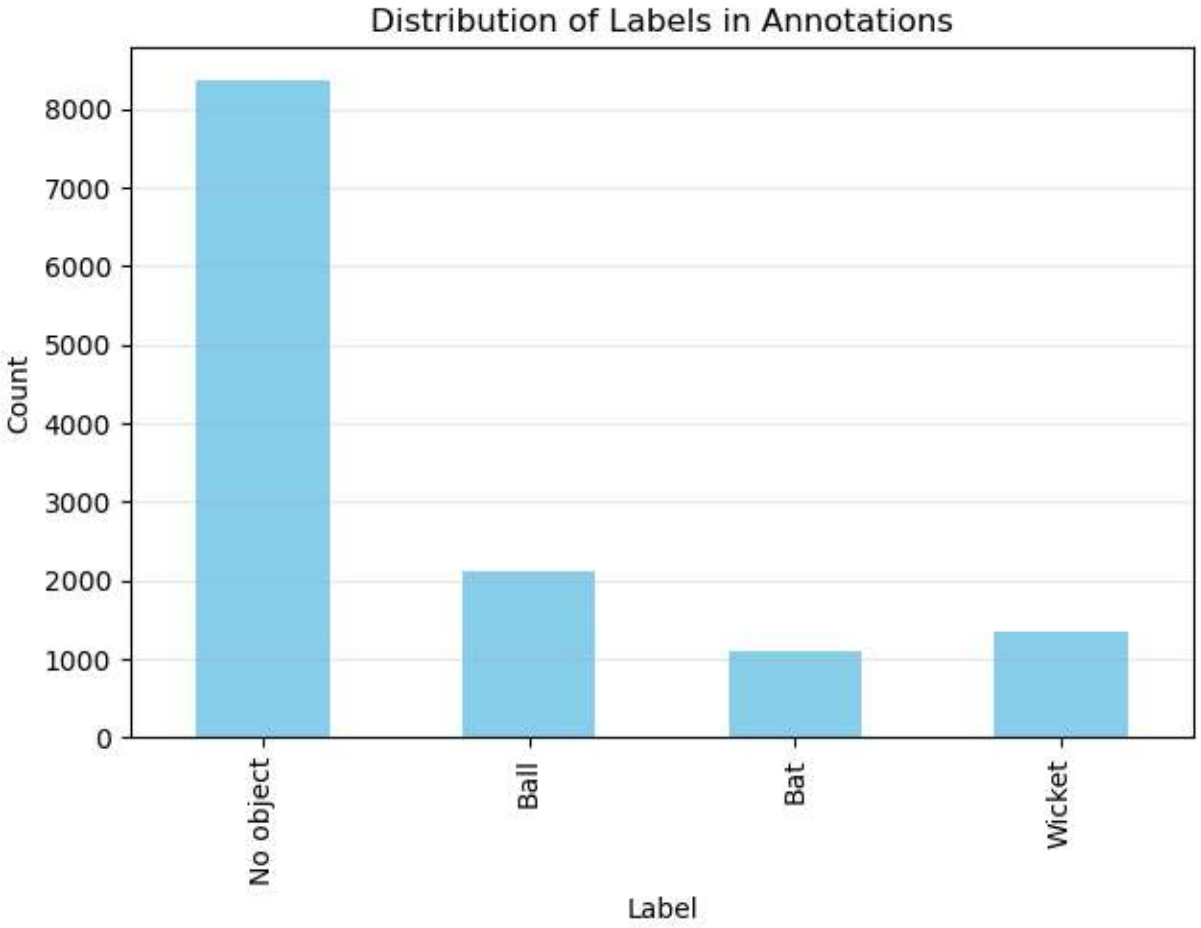
CMInDS
Indian Institute of Technology Bombay

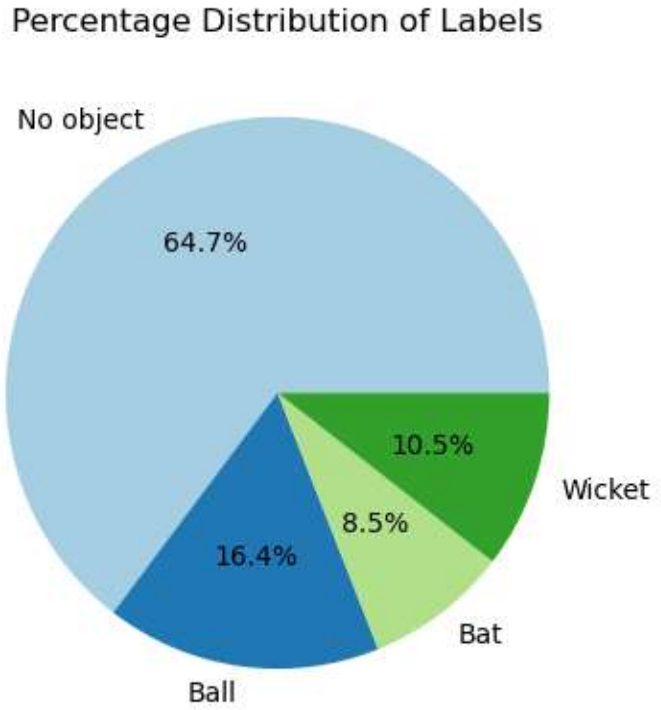


Checking Data Imbalance on Labels

CMInDS
Indian Institute of Technology Bombay

We Checked Data imbalances post adding labels to each cells





Feature Engineering

CMInDS
Indian Institute of Technology Bombay

•Handcrafted features extracted per grid cell:

- HOG (Histogram of Oriented Gradients) for shape/edge cues
- Grayscale intensity & summary statistics for appearance

```
    for r in range(8):
        for c in range(8):
            y1, y2 = r*cell_height, (r+1)*cell_height
            x1, x2 = c*cell_width, (c+1)*cell_width
            cell_img = img[y1:y2, x1:x2]
            features = extract_cell_features(cell_img)
            cell_row, cell_col, label = cell_labels[idx]
            row = {
                'image_name': img_name,
                'cell_row': cell_row,
                'cell_column': cell_col,
                'label': label
            }
            for i, val in enumerate(features):
                row[f'feat_{i}'] = val
            feature_rows.append(row)
            idx += 1
features_df = pd.DataFrame(feature_rows)
print("Extracted features for", len(features_df), "cells")
```

```
# Feature extraction function
def extract_cell_features(cell_img):
    gray = color.rgb2gray(cell_img)
    hog_feat = hog(gray, pixels_per_cell=(8,8), cells_per_block=(1,1), feature_vector=True)
    mean = gray.mean()
    std = gray.std()
    return np.concatenate([hog_feat, [mean, std]])
```

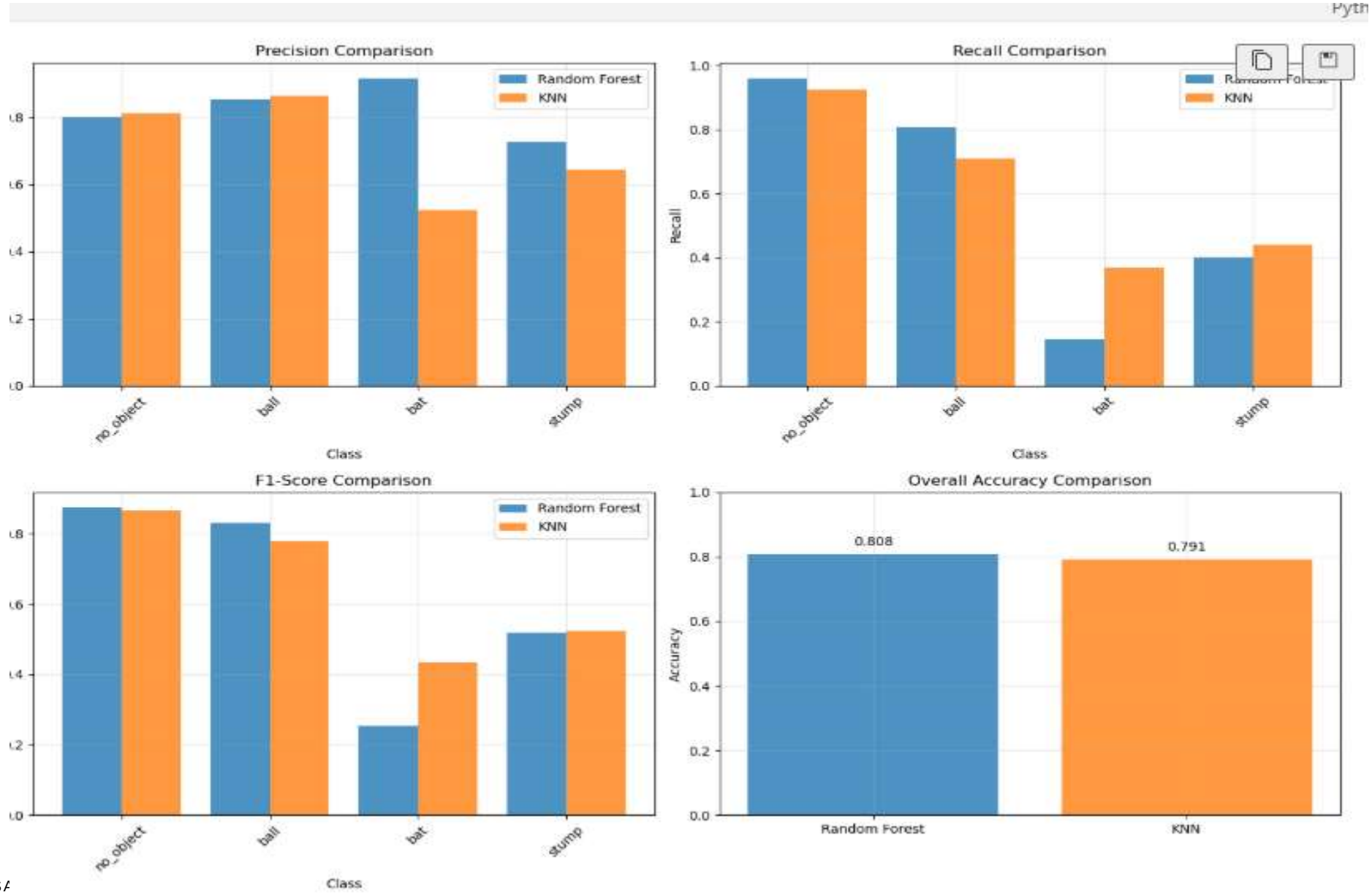
```
1. Why Grayscale?
-----------------
- Simplifies input: reduces RGB channels to a single channel.
- Focuses on shape and structure rather than color.
- Reduces noise from lighting and background variations.

Grayscale conversion is particularly effective for cricket object detection because:
* Cricket objects (bat, ball, stumps) are primarily distinguished by their shape, not color
* Different lighting conditions in cricket matches can cause color variations
* A single-channel representation reduces computational complexity
* Eliminates color-dependent variations while preserving structural information
```

```
+ 2. Why HOG (Histogram of Oriented Gradients)?
+ ----------------------------------------------
+ - Captures edges and contours, which are key for distinguishing objects.
+ - Robust to small rotations and scale changes.
+ - Provides a compact representation of gradient patterns.
+
+ HOG features are ideal for cricket object detection because:
+ * Each cricket object has distinct edge patterns:
+   - Ball: circular edges in all directions
+   - Bat: strong vertical/diagonal edges with elongated shape
+   - Stumps: strong vertical edges with cylindrical structure
+ * HOG descriptors capture these edge orientations effectively
+ * The histogram-based approach makes features robust to minor variations
+ * Works well with traditional machine learning classifiers (SVM, Logistic Regression)
```

Model Training and Evaluation – Random Forest And KNN

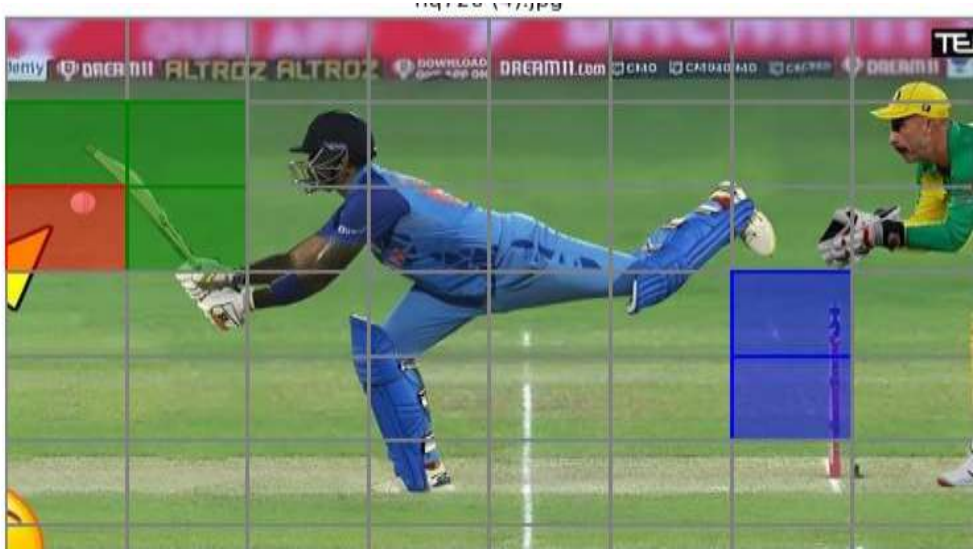
Class	RF Precision	KNN Precision	RF Recall	KNN Recall	RF F1-Score	KNN F1-Score
no_object	0.80	0.81	0.96	0.92	0.87	0.86
ball	0.85	0.86	0.80	0.70	0.83	0.77
bat	0.92	0.53	0.14	0.37	0.25	0.43
stump	0.73	0.64	0.40	0.44	0.51	0.52

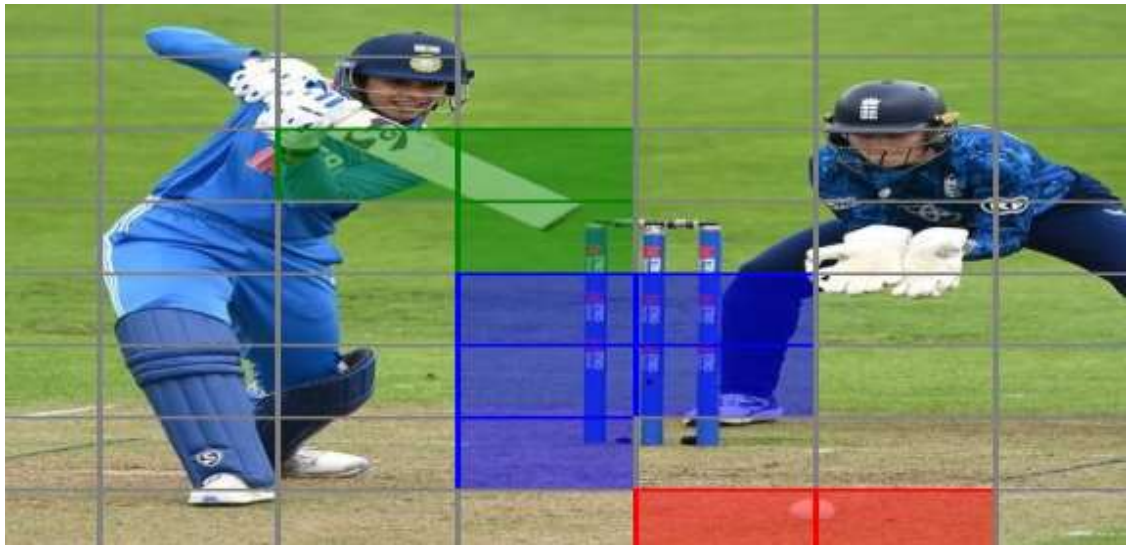
Model Compare Metrics – Random Forest Vs KNN

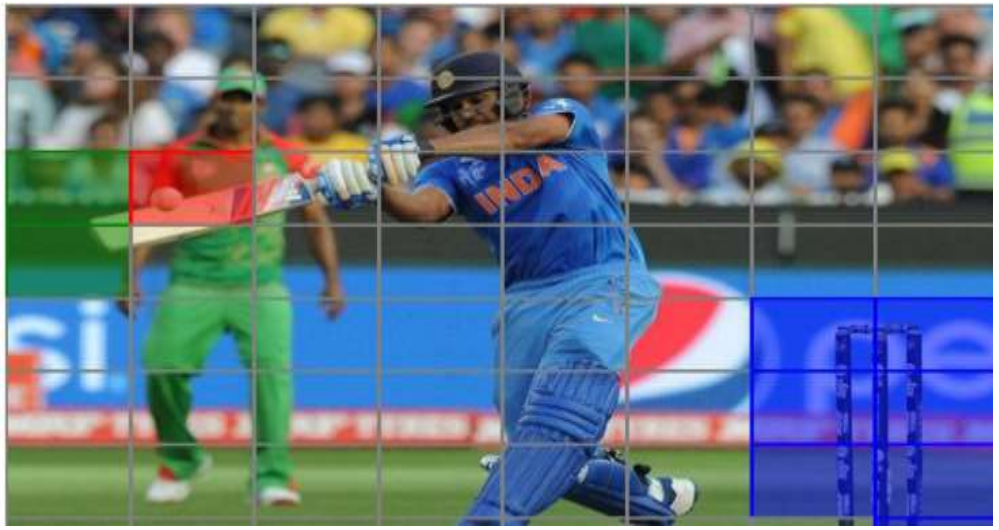


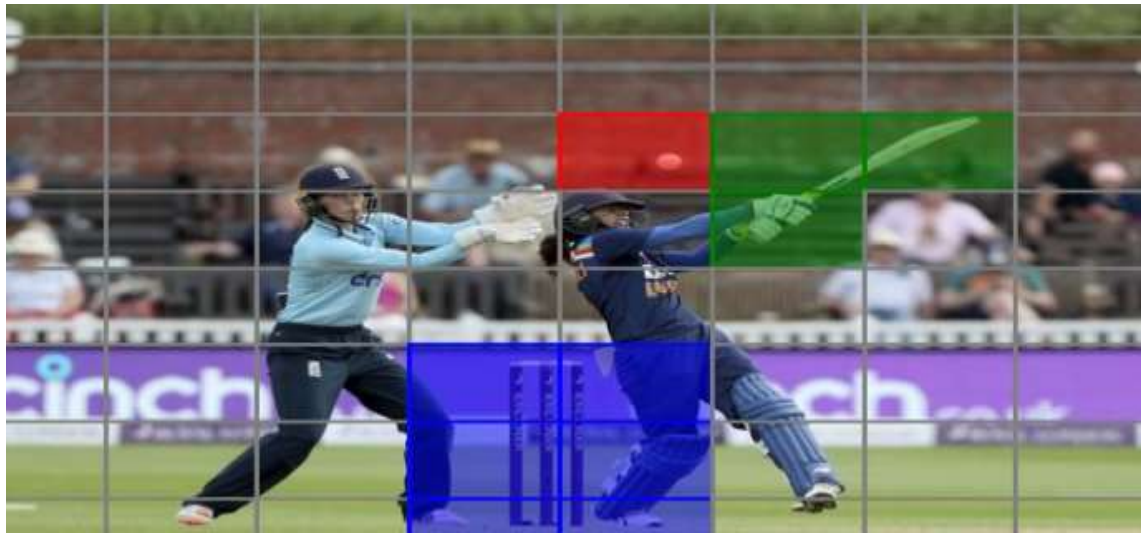
Visualizations

Predicted cell labels are displayed with colored rectangles representing different objects for clear model interpretation.









Learnings and Challenges

CMInDS
Indian Institute of Technology Bombay

Importance of Annotation and Preprocessing

Consistent annotation and preprocessing are critical to achieving reliable model performance and avoiding accuracy drops.

Effective Feature Engineering

Techniques like HOG and grayscale features improve classical machine learning model accuracy through thoughtful selection.

Model Comparison and Trade-offs

Comparing Random Forest and KNN reveals trade-offs between accuracy and computational speed for model selection.

Visualization and Challenges

Visualization aids debugging and understanding, while annotation complexity and scalability remain key challenges.